Transpiler-Architecture Co-Design to Curb Clifford Costs in Fault-Tolerant Quantum Computing

Meng Wang[1,2], Chenxu Liu[2], Samuel Stein[2], Yufei Ding[3], Poulami Das[4], Prashant J. Nair[1], Ang Li[2,5]

[1] *Department of Electrical and Computer Engineering, The University of British Columbia*

[2] *Physical and Computational Science Division, Pacific Northwest National Laboratory*

[3] *Department of Computer Science and Engineering, University of California San Diego*

[4] *Department of Electrical and Computer Engineering, The University of Texas at Austin*

[5] *Department of Electrical and Computer Engineering, University of Washington*

***Abstract*—Quantum Error Correction (QEC) codes form the foundation of Fault-Tolerant Quantum Computing (FTQC) and predominantly use the Clifford+T gate set. Recently, Clifford operations have become the key performance bottleneck in implementing QEC. While state-of-the-art approaches like Pauli-Based Compilation (PBC) reduce Clifford overhead by transforming Clifford gates into Pauli measurements, they do so at the cost of gate-level parallelism, inflating circuit depth and execution times.**

To overcome these limitations, we introduce TACO, a *T*ranspiler–*A*rchitecture *Co*-design framework that tackles the Clifford bottleneck through circuit and architectural optimization. TACO uses FTQC insights to guide hardware-aware Clifford gate elimination and circuit restructuring, and leverages the resulting optimized circuits to refine architectural design. TACO applies FTQC-specific transformations to aggressively reduce Clifford overhead from rotation synthesis and Toffoli decompositions, while preserving gate-level parallelism. The resulting architecture is optimized for the locality and data-movement patterns of these circuits, enabling high-throughput, resource-efficient execution. Our evaluation across diverse benchmarks shows that TACO achieves up to 21.9× (mean 4.4×) reduction in execution time compared to the state-of-the-art baseline.

***Index Terms*—Fault-tolerant quantum computing, quantum error correction, Clifford+T circuits, architecture co-design.**

I. Introduction

Quantum Error Correction (QEC) [18], [36], [46], [47], [66] is essential to enable practical fault-tolerant quantum computing (FTQC). QEC encodes logical qubits into multiple physical qubits to actively detect and correct errors [1], [8], [31], [63]. Among QEC codes, the surface code [30], [42], [46] is especially promising, offering planar connectivity, high thresholds, and efficient decoding, and can realize universal fault-tolerant computation through additional mechanisms such as lattice surgery, code deformation, and magic-state distillation [13], [30], [57]. As quantum algorithms advance, balancing the resource costs of Clifford and T gates in QEC is key to realizing practical FTQC.

Historically, magic-state preparation for T gates dominated FTQC resource costs, often exceeding 95% of the total qubit-cycle volume. This challenge motivated substantial progress in T-gate optimization, significantly reducing T-state overheads [19], [33], [34], [38], [52]. As a result, FTQC resource estimates have begun to expose other costs that were previously treated as negligible. In particular, Clifford operations can now constitute a major, and in some cases dominant, fraction of the computational overhead. As shown in Figure 1, the Clifford-overhead fraction in an 18-qubit QFT circuit increases from 3.2% to 58% under resource estimates spanning the past decade. We observe a similar trend across a broad set of benchmark circuits, as discussed in Section III-A.

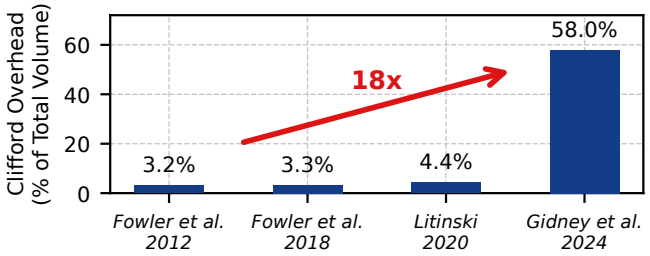
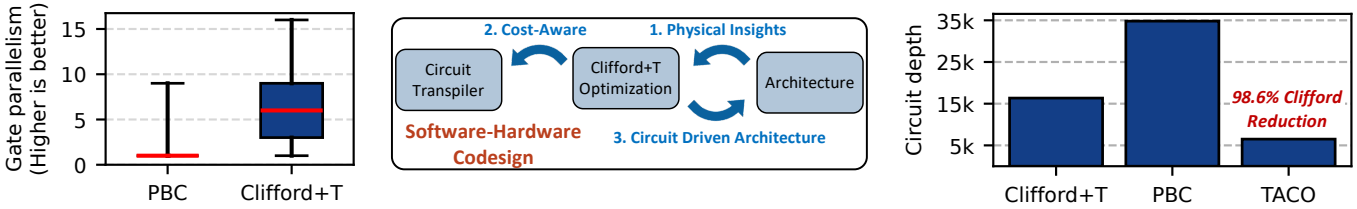


Fig. 1. Clifford gate volume percentage of the 18-qubit QFT circuit execution increased by more than 18× from 3.2% to 58% as T gates become cheaper with improved magic state preparation protocols [29], [30], [34], [52].

Limitations of State-of-the-Art: A common approach for addressing Clifford gate overhead is to convert Clifford+T circuits into Pauli-based circuits (PBC) [15], [51]. This transformation commutes all Clifford gates to the end of the circuit, effectively absorbing them into the final measurements and eliminating explicit Clifford operations. However, this approach comes with important trade-offs. PBC introduces numerous multi-qubit $\frac{\pi}{4}$ rotations. These rotations significantly constrain gate parallelism (i.e., the number of gates that can be executed in parallel), as shown in Figure2 (a). Moreover, as multi-qubit logical operations involve more qubits, their error rates rise, often requiring higher code distances to maintain fault tolerance. This, in turn, increases physical qubit requirements and further limits the efficiency of PBC.

Our Proposal: We address the Clifford bottleneck with TACO, a systematic **T**ranspiler–**A**rchitecture **C**o-design **O**ptimization framework. We observe that optimizing either the circuit or hardware in isolation cannot efficiently resolve the trade-offs between gate count, parallelism, and hardware cost in FTQC. TACO overcomes these challenges through three tightly connected strategies: (1) *Hardware-informed Clifford optimization* removes Clifford gates by analyzing their origins and exploiting hardware-native operations. This re-



(a) Gate parallelism comparison. (b) TACO: Software-Hardware codesign. (c) Depth of original and optimized circuits.

Fig. 2. Comparison of Clifford optimization approaches showing gate reduction vs. circuit depth trade-offs for an 18-qubit QFT circuit. (a) The state-of-the-art Clifford optimization method, PBC, significantly restricts gate parallelism. (b) TACO employs a software-hardware co-design approach that (c) reduces Clifford gates by 98.6% in the QFT circuit, which is comparable to PBC, while achieving 5.3× lower circuit depth.

duces Clifford count by over 91% on average while maintaining parallelism. (2) *FTQC-aware transpilation* leverages the simplified Clifford structure to map circuits into fault-tolerant forms that align with the true hardware cost hierarchy. This ensures both Clifford and T gates are synthesized efficiently. (3) *Resource-locality-driven architecture* exploits the locality and regularity in optimized circuits to allocate compute and storage resources efficiently. This helps sustain high gate throughput and minimizes physical qubit overhead.

Insight 1: Hardware-Informed Clifford+T Optimization: We use the insight that Clifford gates in FTQC circuits mainly arise from two sources: the decomposition of Toffoli gates and the synthesis of single-qubit rotations such as R_z. In typical quantum algorithms, each Toffoli gate decomposes into 8 Clifford and 7 T gates. We show that these Clifford gates can be merged or canceled *locally* based on algebraic structure, without reducing gate parallelism. For single-qubit rotations, standard Clifford+T synthesis can expand each rotation into hundreds of gates, with Cliffords comprising up to 60% of the sequence. Unlike the Toffoli case, these Clifford gates are not locally removable within Clifford+T, since they serve as basis changes between T gates ($R_z(\frac{\pi}{4})$). However, recognizing that $R_x(\frac{\pi}{4})$ can be implemented using mechanisms analogous to T gates expands the effective gate set, making most basis changes unnecessary and reducing Clifford gates by over 98%. By combining local Clifford gate cancellation for Toffoli decompositions with a hardware-native implementation of single-qubit rotations, we eliminate over 98.6% of Clifford gates. As shown in Figure 2(c), our optimized Clifford+T circuits achieve a 5.3× lower circuit depth compared to PBC.

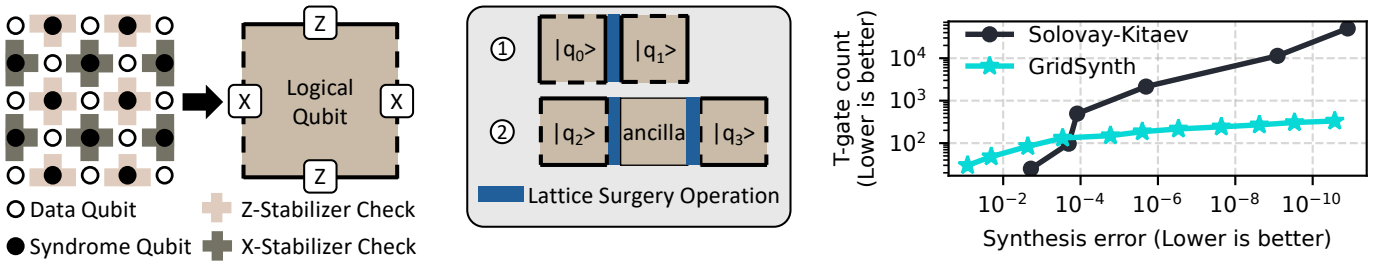
Insight 2: FTQC-Aware Circuit Transpilation: Transpiling algorithmic circuits into Clifford+T form involves two key steps: decomposition and gate synthesis. Although decomposing circuits into standard gate sets may appear straightforward, we find that it offers significant, often hidden optimization opportunities that are largely missed by existing transpilers [6], [43], [57], [72]. Our FTQC-aware transpiler exploits these opportunities by jointly coordinating decomposition and synthesis to minimize the resulting Clifford+T gate count.

Insight 3: Resource-Locality-Driven Architecture: We exploit the high spatial and temporal locality of our optimized FTQC circuits, especially in extended $\frac{\pi}{4}$ rotation sequences, to design an architecture with dedicated compute and memory blocks. Compute qubits, characterized by frequent gate activity, are clustered in compute blocks optimized for rapid access to magic state resources and efficient execution of consecutive $\frac{\pi}{4}$ rotations. These blocks are configured to expose both the X and Z edges of each logical qubit, enabling flexible, low-latency lattice surgery with state-distillation ancillae. Storage qubits, which remain idle except during Clifford operations, are assigned to memory blocks arranged in a compact three-row layout. This layout minimizes physical area while preserving connectivity via shared ancilla tiles. This enables rapid (typically within a few QEC cycles) and flexible transfer of logical qubits between memory and compute blocks.

At a higher level, our approach systematically integrates hardware layout, circuit transpilation, and architectural refinement through a cross-layer feedback loop. Hardware layout decisions, such as compute and memory block partitioning and ancilla tile placement, directly inform transpiler optimizations. This allows Clifford+T circuits to be mapped to maximize locality and throughput. Conversely, insights from circuit transpilation, such as $\frac{\pi}{4}$ rotation clustering and gate reuse, drive targeted refinements in the hardware design. This iterative co-design allows each layer to reinforce the others, resulting in a 2.3× reduction in the total Clifford+T gate count. From the resulting circuit, we further eliminate, on average, 91% of remaining Clifford gates while preserving parallelism, leading to a geometric mean of 4.4× speedup in QEC cycles. Our resource-locality-driven architecture sustains one logical gate per QEC cycle with just $1.5n + 4$ logical qubit tiles. This approach significantly outperforms prior designs that require $2n + \sqrt{8n} + 1$ logical qubit tiles [51].

In summary, this work has *four* key contributions:

- We identify Clifford gates as the dominant, yet previously overlooked source of overhead in FTQC, contributing to more than half of the total gate overhead.
- We propose TACO, a cross-stack framework that reduces Clifford gate overhead by 91% on average (up to 98.6%). This leads to a mean 4.4× speedup across diverse benchmarks.
- We develop an FTQC-aware transpiler that reduces overall gate count by 2.3× compared to state-of-the-art compilers.
- We design a hardware architecture tailored to the structure and locality of optimized circuits, achieving one logical gate per QEC cycle with only $1.5n+4$ logical qubit tiles. This approach substantially outperforms prior designs that require $2n+\sqrt{8n}+1$ logical qubit tiles.



(a) Distance-3 surface code layout. (b) Lattice surgery between logical qubits. (c) Comparison of FTQC gate synthesis algorithms.

Fig. 3. (a) Distance-3 surface code layout showing data qubits (○), syndrome qubits (●), stabilizers (shaded areas), and logical qubit abstraction. X and Z-stabilizers are shown with different colors. (b) Lattice surgery operations between ① neighboring logical qubits and ② non-neighboring logical qubits using an ancilla logical qubit. (c) T-gate count versus synthesis error when synthesizing a random 1-qubit unitary into a Clifford+T gate sequence using the Solovay-Kitaev algorithm [24], [45] and GridSynth [64].

II. Background

A. Surface Code and Fault-Tolerant Operations

Basics of Surface Code: Quantum error correction (QEC) protects fragile qubits by encoding logical qubits into multiple physical qubits. The surface code [16], [26], [30] arranges qubits in a 2D lattice with data qubits (○) and syndrome qubits (●) that perform stabilizer measurements to detect errors (Figure 3(a)). Each surface code patch defines a logical qubit with distance d, capable of correcting up to $\frac{d-1}{2}$ errors.

Lattice Surgery: Logical fault-tolerant operations are implemented via *lattice surgery* [27], [29], [42], [70], which enables qubit interactions by merging or splitting patches along shared edges (Figure 3(b)). For non-adjacent qubits, interactions are facilitated by repositioning or using ancillary qubits.

Gate Implementation: The surface code natively supports most Clifford gates (Pauli-X/Y/Z, Hadamard, CNOT) [30], [51], while the Phase gate (S) requires code deformations or gate teleportation with $|Y\rangle$ states. The non-Clifford T gate, essential for universal computation, is implemented through gate teleportation using magic states $|M\rangle$ prepared via resource-intensive magic state distillation [12], [34], [44], [52].

B. Qubit-Cycle Cost of FTQC

The overhead of FTQC execution is quantified using the *qubit-cycle volume* metric, where *qubit* denotes physical qubits and *cycle* is measured in QEC cycles. Each cycle represents one round of syndrome measurement and error correction, and the total volume is the product of these quantities. The FTQC circuit size determines the total number of logical qubits. In contrast, the number of physical qubits per logical qubit depends on the chosen code distance (d), set by the required logical error rate and total number of cycles.

The circuit's critical path determines the total number of QEC cycles. Within the Clifford+T gate set, the execution cost of each gate type differs. Pauli gates can be applied with no cost [30]. CNOT gates use lattice surgery and require *3d+4* cycles (where d is the code distance). Hadamard gates require patch-deformation techniques, requiring *3d+4* cycles. The S gate requires preparing and injecting a $|Y\rangle$ state, with the total cost, including both preparation and consumption, taking *1.5d+3* cycles. For the T gate, once a magic state is prepared, consuming it takes *2.5d+4* cycles. These cost estimates are based on the recent resource estimation study [7].

C. Fault Tolerant Quantum Computing: Clifford+T Gates

To execute an FTQC circuit, all gates must be expressed (transpiled) in the Clifford+T basis. Some gates, such as the Toffoli, can be exactly decomposed into Clifford+T gates. However, many gates, especially single-qubit rotations, do not have an exact representation and instead require approximate synthesis into sequences of Clifford+T gates.

The Solovay-Kitaev algorithm [24], [45] was one of the first methods proposed for this synthesis task, offering asymptotically efficient approximations for arbitrary unitary gates. The general algorithm can accept any unitary and generate approximations in any universal gate set that includes inverses for all its gates. It produces a sequence of length $O\left(\log^c(1/\epsilon)\right)$ to achieve an approximation error ϵ, with practical $c \approx 3.97$ [24], where c is a constant. Thus, the resulting gate sequences are prohibitively long even for moderate accuracy.

Recent number-theoretic approaches such as GridSynth [64] have dramatically improved synthesis efficiency by specifically targeting single-qubit Z-axis rotations (Rz gates). As shown in Figure 3(c), GridSynth achieves an error below 10^{-10} with just 332 T gates, compared to over 50,000 gates using Solovay-Kitaev. Since any quantum algorithm can be decomposed into Rz plus Clifford gates, GridSynth is adopted as the default synthesis method in our framework, ensuring both accuracy and practical gate counts for large-scale FTQC circuits.

III. Motivation

A. The Decadal Shift: T-Gate Costs Down, Clifford Costs Up

Historically, T gates were the primary driver of FTQC overhead, with magic state preparation requiring over 1000× more resources than Clifford operations [13], [30]. This motivated a decade of advances in T-gate and magic-state optimization [29], [52]. As shown in Figure 4, recent progress in magic-state cultivation [34] has reduced this overhead by more than 100×, bringing T-gate costs close to those of logical CNOT operations. This shift also reflects an Amdahl's law effect: as the T-gate component is accelerated, the remaining Clifford component increasingly limits end-to-end improvement. Thus, once magic-state overhead becomes comparable to

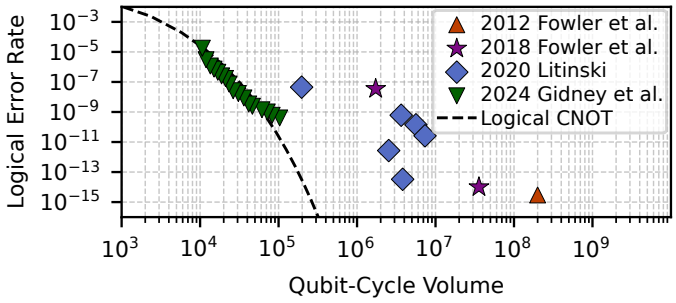


Fig. 4. Magic state preparation overhead (qubit-cycle volume) has decreased by more than 100× since 2012 and is now comparable to logical CNOT operations. Data from [29], [30], [34], [52].

logical Clifford operations, further T-only optimizations yield diminishing returns unless Clifford overhead is also reduced.

For the benchmarks in Table I (see Section V-C for details), which cover key FTQC algorithms, Clifford gates now account for 58–65% of execution overhead, with an average of 60.5%. From an Amdahl's law perspective, even eliminating the T-attributable portion entirely would provide only a bounded speedup of roughly 1.7×–2.0× in this regime. As T-gate costs continue to fall, Clifford overhead, once considered negligible, has emerged as a dominant challenge to scaling FTQC.

B. The Parallelism Trade-Off for Tackling Clifford Overhead

A common method for reducing Clifford overhead is Pauli-Based Compilation (PBC), which commutes all Clifford gates to the end of the circuit and absorbs them into the final measurements. While this eliminates explicit Clifford gates, it comes at the significant cost of reduced gate-level parallelism.

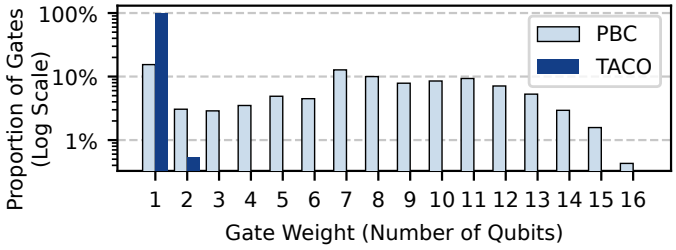


Fig. 5. Gate-weight distribution of PBC and TACO for the 20-qubit QFT circuit. PBC produces multi-qubit operations involving up to 16 qubits, while TACO keeps all operations at one- or two-qubit weight. This preserves high gate parallelism by avoiding wide, multi-qubit gates.

Commuting single-qubit Clifford gates through non-Clifford gates keeps non-Clifford operations localized to individual qubits, though in altered forms. However, when two-qubit Clifford gates are commuted through non-Clifford gates, these operations become entangled with additional qubits, resulting in multi-qubit rotation gates. This process compounds as more two-qubit Cliffords are moved, ultimately generating operations that act on an increasing number of qubits, i.e., with a higher weight. Figure 5 shows the gate-weight distribution from PBC for the 20-qubit QFT circuit. Starting from only single- and two-qubit gates, PBC generates high-weight operations involving up to 16 qubits.

This effect is particularly problematic for quantum algorithms, which depend on multi-qubit entanglement. After PBC transformation, most non-Clifford gates act on large subsets of qubits, severely restricting parallel execution opportunities. As shown in Figure 2(c), the 40,777 gates produced by PBC for the 18-qubit QFT circuit yield a circuit depth of 37,756, averaging just 1.08 gates per circuit layer.

C. Preserving Parallelism with Structured Clifford+T Circuits

A key challenge in reducing Clifford overhead without losing parallelism lies in how CNOT gates are treated. By deliberately avoiding the commutation of CNOT gates through the circuit, one can preserve the natural gate parallelism critical for efficient quantum computation. Thereafter, one can then focus on optimizing the remaining components, namely Toffoli gates and single-qubit gate sequences, and eliminating Clifford gates within these sequences. This helps target the primary sources of Clifford overhead while maintaining parallelism.

To optimize single-qubit gate sequences, we use the Matsumoto-Amano (MA) Normal Form [35], [55], a canonical structure for any single-qubit Clifford+T sequence:

$$\text{MA Normal Form} := (T|\epsilon)\,(HT|SHT)^*\,C \tag{1}$$

In this form, $(T|\epsilon)$ is an optional initial T gate, followed by a sequence of HT or SHT patterns (denoted by $(HT|SHT)^*$), ending with a Clifford gate C. Notably, the MA Normal Form guarantees both a minimal T-gate count and a unique decomposition for any target unitary, enabling systematic and efficient optimization of these sequences. By converting all single-qubit gate sequences into MA Normal Form, one can isolate Clifford reduction to two subproblems. First, eliminating redundant Cliffords within Toffoli decompositions, and second, those within these structured single-qubit sequences. This targeted strategy enables effective Clifford gate removal while preserving circuit parallelism. As shown in Figure 5, TACO limits operation weight to at most two qubits.

IV. Design of TACO

TACO is a full-stack optimization framework for FTQC. As shown in Figure 6, TACO unifies dynamic circuit decomposition, Clifford gate reduction, and architecture-aware optimization in a single workflow. We begin by presenting techniques for Clifford gate reduction, both for single-qubit sequences (Section IV-A) and CCX (Toffoli) gate decompositions (Section IV-B). Next, we introduce a dynamic circuit transformation pass (Section IV-C) that efficiently minimizes high-cost gates during synthesis, leveraging circuit structure for further reductions. Finally, we show how the locality patterns emerging from these optimized circuits directly inform the design of a tailored FTQC architecture (Section IV-D).

A. Structured Clifford Reduction for Single-Qubit Gates

TACO systematically identifies all single-qubit gate sequences in the circuit and converts them into Matsumoto-Amano (MA) Normal Form (Equation 1). This structured representation enables TACO to apply Clifford reduction to each sequence, effectively eliminating most Clifford gates

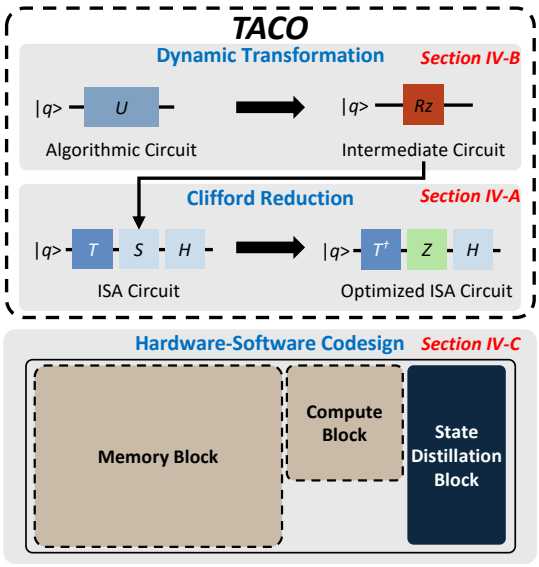


Fig. 6. Overview of the design. TACO begins with dynamic gate decomposition of the algorithmic circuit, transforming it into an intermediate form with reduced Rz gates before synthesizing into a Clifford+T circuit. TACO then applies Clifford reduction to minimize execution time. The resulting circuit's high locality of $\pi/4$ rotations informs a software-guided hardware design.

while preserving circuit integrity. The approach is shown below with a representative gate sequence:

$$T\ S\ H\ T\ H\ T\ S\ H\ T \tag{2}$$

1) Eliminating Phase Gates: In MA Normal Form, only two fundamental gate patterns can appear – HT and SHT, which result in four possible gate patterns in the sequence:

$$HT\ HT;\ HT\ SHT;\ SHT\ HT;\ SHT\ SHT \tag{3}$$

A key observation is that *every S gate in these sequences is immediately preceded by a T gate*, with a single exception: an initial SHT prefix. An equivalence exists between TS gates and T†Z gates, which we can verify through:

$$TS = \begin{bmatrix}1 & 0\\ 0 & e^{i\frac{\pi}{4}}\end{bmatrix}\begin{bmatrix}1 & 0\\ 0 & i\end{bmatrix} = \begin{bmatrix}1 & 0\\ 0 & e^{i\frac{\pi}{4}i}\end{bmatrix} = \begin{bmatrix}1 & 0\\ 0 & e^{i\frac{3\pi}{4}}\end{bmatrix} \tag{4}$$

$$T^\dagger Z = \begin{bmatrix}1 & 0\\ 0 & e^{-i\frac{\pi}{4}}\end{bmatrix}\begin{bmatrix}1 & 0\\ 0 & -1\end{bmatrix} = \begin{bmatrix}1 & 0\\ 0 & -e^{-i\frac{\pi}{4}}\end{bmatrix} = \begin{bmatrix}1 & 0\\ 0 & e^{i\frac{3\pi}{4}}\end{bmatrix} \tag{5}$$

Note that T† and Z gates are natively supported on hardware, with Z gates being executed virtually. With this equivalence, we can effectively transform all expensive S gates into 'free' Z gates. The sample gate sequence shown in Equation 2 can be transformed into the new sequence shown in Figure 7.

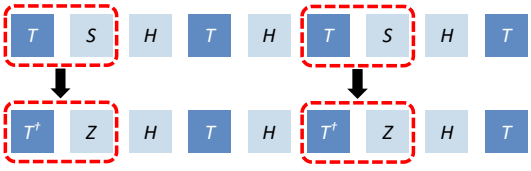


Fig. 7. Remove Phase (S) gate using the identity: $TS = T^\dagger Z$.

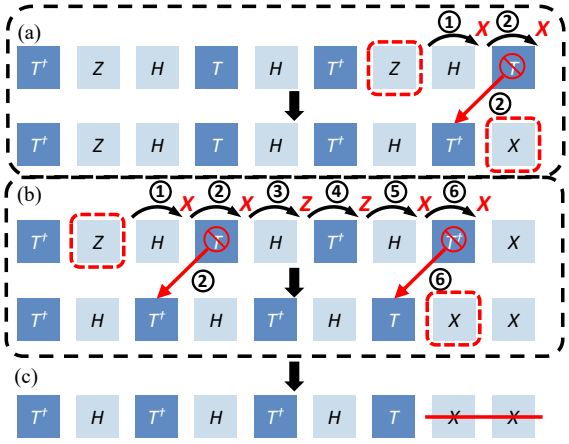


Fig. 8. Commute and merge Pauli gates using commuting rules defined in Equation 6. (a,b) shows the commuting process of the two Pauli gates and (c) the merging of the Pauli gates. Circled numbers are the step order.

2) Eliminating All Pauli Gates: Once we eliminate all S gates from the sequence, we are left with H, T, T†, and Z gates. Although Z gates can be executed virtually, we want to eliminate them first to make the next step of eliminating H gates easier. The following commutation relations hold:

$$\begin{aligned} ZH &= HX;\ XH = HZ\\ ZT &= TZ;\ ZT^\dagger = T^\dagger Z\\ XT &= T^\dagger X;\ XT^\dagger = TX \end{aligned} \tag{6}$$

These relations enable all the Pauli gates to be commuted at the end of the sequence and merged as shown in Figure 8.

3) Eliminating All Hadamard Gates: To this point, the remaining gates in the sequence are H, T, and T† with a potential Pauli gate at the end of the sequence. Unlike Pauli Gates, Hadamard cannot be easily commuted to the end of the sequence. To further eliminate Hadamard gates, we introduce a new gate operator: $Rx(\frac{\pi}{4})$. This operator requires the same hardware resources and implementation complexity as the T gate, with the only distinction being that the T gate performs lattice surgery between a magic state and the Z edge of the target qubit, whereas $R_x\left(\frac{\pi}{4}\right)$ operates on the X edge [42], [51]. The following commutation relation holds:

$$HT = Rx(\frac{\pi}{4})H,\quad HT^\dagger = Rx^\dagger(\frac{\pi}{4})H,\quad HH = I \tag{7}$$

We iterate over the sequence from left to right to efficiently remove all Hadamard gates. When encountering an H, we commute it forward using Equation 7. The two cancel if we meet a second H. This process is repeated until all H gates are pushed to the end or eliminated. Figure 9 illustrates this.

4) Transpilation Efficiency: The three FTQC-specific optimizations, elimination of Phase (Section IV-A1), Pauli (Section IV-A2), and Hadamard gates (Section IV-A3), each run in linear time $O(n)$, where n is the number of gates in the circuit.

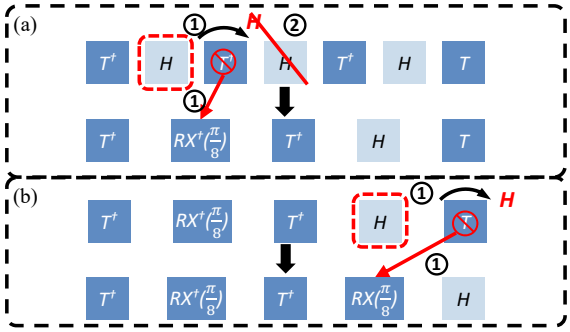


Fig. 9. Commuting all the Hadamard gates to the end of the sequence using the commuting rules in Equation 7. Circled numbers indicate the step order.

These passes are highly efficient in practice. For example, our 18-qubit QFT benchmark (see Section VI-C) contains 459 R_z gates and expands to 104,217 Clifford+T gates after synthesis. TACO completes the entire transpilation in under 1 second, compared to 16.1 seconds required by GridSynth for gate synthesis alone. Thus, TACO introduces negligible overhead and imposes no FTQC compilation bottleneck.

5) Exploiting Locality in Optimized Circuits: Following the three Clifford+T gate sequence optimizations, the resulting circuit displays a high degree of **locality**. That is, multiple $R\left(\frac{\pi}{4}\right)$ rotations are often applied consecutively to the same qubit. Because each such rotation requires interaction with a magic state, this structural locality can be exploited to improve execution efficiency by strategically placing qubits near magic-state sources. This intrinsic property of the optimized circuit directly informs our architecture design in Section IV-D, enabling more efficient resource allocation and throughput.

B. Parallelism-Preserving Clifford Reduction in Toffoli Gates

A single Toffoli gate decomposes into seven $T/T^\dagger$ gates, six CNOTs, and two Hadamard gates, as shown in Figure 10(a). Figure 10(b) illustrates the commutation process of the first non-Clifford gate ($T^\dagger$). ①: the $T^\dagger$ gate is represented as $R_Z(-\pi/4)$. ②: it is then commuted through a CNOT gate, and ③: subsequently through a Hadamard gate.

Figure 10(c) shows the circuit after all remaining non-Clifford gates have been commuted to the front, leaving the Clifford gates grouped afterward and labeled C_1 through C_8. These Clifford gates can be locally canceled. Gates C_3 and C_4, both CNOTs acting on the same target qubit, commute and can be swapped. After this swap, the consecutive self-inverse pairs (C_2, C_4), (C_3, C_5), and (C_7, C_8) cancel, followed by the remaining Hadamard pair (C_1, C_6).

After these local cancellations, only the non-Clifford rotations acting on the three qubits of the original Toffoli gate remain, as shown in Figure 10(d). All operations stay confined to this local three-qubit subspace, ensuring that no additional commutation steps or circuit depth overhead are introduced.

C. FTQC-oriented Dynamic Circuit Transformation

A key step in our workflow is an intermediate circuit transformation, motivated by the fact that efficient gate synthesis algorithms such as GridSynth [64] operate on a restricted gate

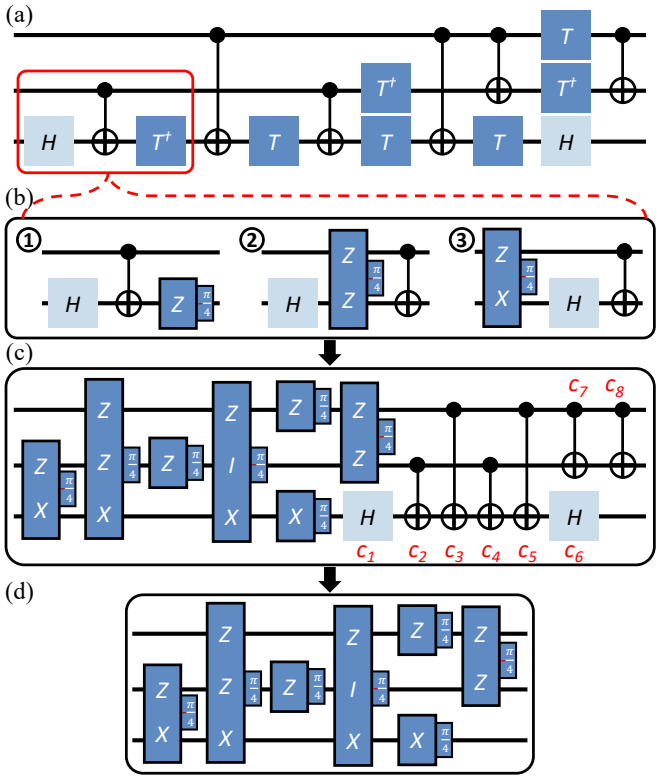


Fig. 10. (a) Decomposition of the Toffoli gate into Clifford and non-Clifford components. (b) Commutation of the first non-Clifford gate ($T^\dagger$): ① $T^\dagger$ represented as $R_Z(-\pi/4)$, ② commuted through a CNOT, and ③ through a Hadamard gate. (c) The circuit obtained after commuting all non-Clifford gates to the front. (d) Eight Clifford gates cancel pairwise after swapping C_3 and C_4. Consecutive self-inverse Clifford pairs (C_2, C_4), (C_3, C_5), and (C_7, C_8) cancel first, followed by the remaining pair (C_1, C_6).

set. Reducing the number of gates that require synthesis at this stage directly impacts overall resource cost. To highlight this challenge, we evaluated common NISQ-oriented transpilers on two 4-qubit Quantum Phase Estimation (QPE) circuits. As shown in Figure 11, no configuration consistently achieves low R_z gate counts for practical FTQC needs, revealing an important gap in current transpiler strategies.

To address this, we develop a dynamic transformation method optimized for FTQC that consists of three stages. First, we study all possible decomposition rules for gates up to three

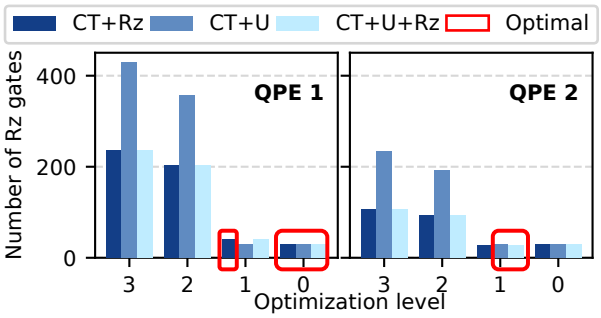


Fig. 11. Number of Rz gates in intermediate circuits obtained from transpiling two 4-qubit QPE circuits using Qiskit Transpiler across optimization levels 0-3 and three basis gate sets (Clifford+T+U, Clifford+T+Rz, Clifford+T+U+Rz). Lower Rz gate counts indicate better optimization.

qubits, selecting the option with the minimum FTQC cost for each. Second, we simplify trivial cases by replacing rotation gates that are Clifford+T equivalent, such as mapping $R_z(\pi)$ to a Pauli-Z. Third, we merge consecutive single-qubit gates whenever possible, leveraging local cancellations to further reduce the gate count. This targeted approach yields optimal gate counts on both QPE benchmarks, consistently outperforming NISQ-oriented transpiler configurations and demonstrating the value of FTQC-specific dynamic transformation.

D. Architecture Co-Design Guided by Circuit Locality

The optimized Clifford+T circuits from Section IV-A exhibit high $\pi/4$ rotation locality – that is, long sequences of $R\left(\frac{\pi}{4}\right)$ rotations are repeatedly applied to the same qubit. The length of these sequences can span dozens or even hundreds of consecutive rotations. To exploit this structural regularity, we propose an FTQC architecture consisting of two logical regions: a *compute block* and a *memory block*. The compute block hosts logical qubits that undergo frequent $\pi/4$ rotations, enabling efficient interaction with magic-state resources. The memory block holds the remaining qubits in a compact, area-efficient layout while still supporting necessary Clifford operations such as CNOT and Hadamard gates.

1) Compute Block: The compute block is optimized for executing long sequences of $\frac{\pi}{4}$-rotation operations. In the Clifford-Reduced circuit from Section IV-A, there are four types of $\frac{\pi}{4}$ rotations: Z-axis rotations, X-axis rotations, and their inverses. The $Rz(\frac{\pi}{4})$ gate, equivalent to the T gate, requires lattice surgery between a magic state qubit and the Z-edge of the target qubit. Similarly, the $Rx(\frac{\pi}{4})$ gate requires interaction with the X-edge of the target qubit.

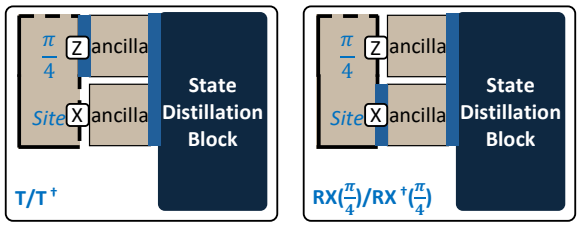


Fig. 12. Layout of the compute block, showing the $\frac{\pi}{4}$ logical qubit site with both X and Z edges exposed via ancilla qubits to the state distillation block. This configuration supports $Rz(\frac{\pi}{4})$ (left) and $Rx(\frac{\pi}{4})$ rotations (right) by enabling selective edge interaction. One compute block uses four logical qubit tiles: two for the $\frac{\pi}{4}$-site qubit and two ancilla tiles.

As shown in Figure 12, each compute block includes a $\pi/4$ logical qubit site with X and Z edges exposed through ancilla tiles connected to the state distillation block. These edges are activated according to the required rotation axis. During magic-state injection, the sign of the resulting $R(\frac{\pi}{4})$ rotation is probabilistic. If the opposite sign is produced, the intended rotation is recovered using a Clifford correction, which is unavoidable but much cheaper than another non-Clifford rotation. High $\pi/4$-rotation locality ensures that each target qubit remains in the compute block throughout its rotation sequence, avoiding unnecessary movement between memory and compute regions and improving execution efficiency.

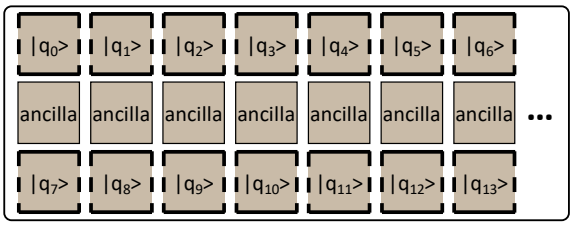


Fig. 13. Memory block layout for storing idle qubits. Logical qubits occupy the first and third rows, while the middle row is reserved for ancilla qubits.

2) Memory Block: The memory block stores the remaining logical qubits, which are mostly idle except during the execution of CNOT or Hadamard gates. We adopt the compact tile layout from [51], illustrated in Figure 13. The design consists of three rows: logical qubit tiles occupy the first and third rows, while the middle row contains ancilla tiles. This arrangement requires only $1.5n$ tiles for n logical qubits and supports direct interactions between any qubit pair via the shared ancilla layer, enabling flexible and efficient CNOT operations. Hadamard gates can be applied locally with minimal overhead. The central ancilla row is also connected to the compute block, enabling seamless movement of qubits between memory and compute regions as needed for fault-tolerant execution.

3) Patch Rotation: In the memory block, each logical qubit exposes only a single computational basis for interactions. When operations require the complementary basis, a *patch rotation* is performed: 1 cycle to expand the patch, one cycle to rotate the logical basis, and one cycle to shrink back, totaling 3 code cycles [51]. Since these rotations are infrequent, they do not impact overall performance.

4) Qubit Transfer Between Memory and Compute Blocks: When a logical qubit moves between memory and compute blocks, the architecture provides direct access from any memory location to the compute block. The transfer completes in a single code cycle by expanding the qubit into the compute block and contracting it back to its operational configuration.

E. Optimizing Compute and Distillation Block Placement

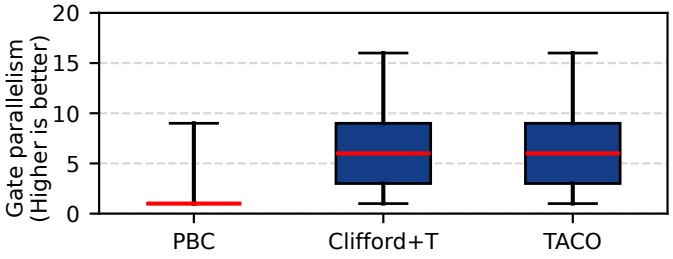


Fig. 14. Gate parallelism comparison of 20-qubit QFT. TACO preserves the same high gate-level parallelism as the original Clifford+T circuit.

As shown in Figure 14, TACO-optimized circuit preserves the high gate-level parallelism of the original Clifford+T circuit. This parallelism requires multiple magic states within each circuit layer, which necessitates multiple distillation blocks. This raises two practical questions: how many distillation blocks are optimal, and where should they be placed.

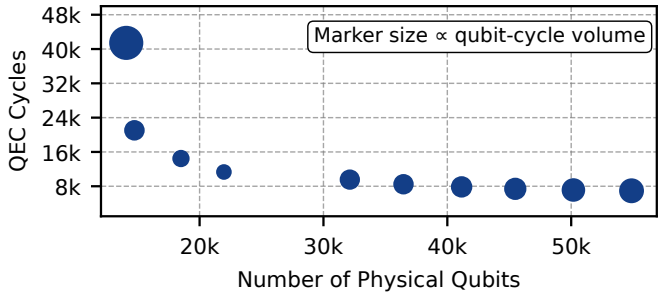


Fig. 15. Qubit-cycle comparison for 18-qubit QFT execution with magic state preparation throughputs from 1-10 magic states/cycle using Magic State Cultivation [34]. Marker sizes are proportional to qubit-cycle volume, with four magic states/cycle being the optimal configuration.

Minimizing Space-Time Volume via Factory Scaling. The primary architectural objective of TACO is to minimize the total space-time volume. This requires balancing the physical qubit overhead of magic state factories against the resulting increase in circuit throughput. Unlike prior heuristics that suggest fixed overhead ratios for distillation, we explicitly model the physical cost of each distillation block based on its surface code distance and layout requirements. Figure 15 illustrates the QEC cycles required for an 18-qubit QFT circuit as a function of available magic state blocks. The marker size indicates the total qubit-cycle volume, which incorporates the actual spatial footprint of both memory qubits and the allocated distillation factories. We observe that as throughput increases, the cycle count decreases significantly, reaching an optimal volume at 4 magic-state blocks. This configuration represents a 57% reduction in volume compared to a single-block setup. Beyond this point, further scaling yields diminishing returns because the added spatial overhead outweighs the marginal cycle reductions. TACO automatically identifies this optimal configuration by simulating these trade-offs, ensuring that resource allocation is driven by total volume minimization rather than arbitrary overhead limits.

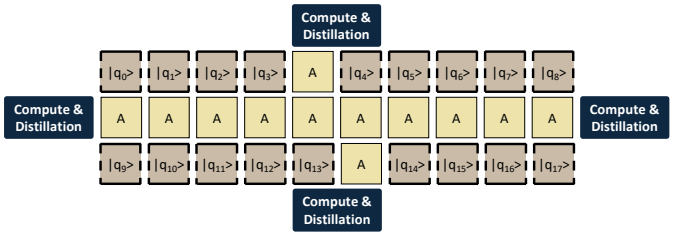


Fig. 16. Architecture for 18-qubit QFT execution featuring 18 logical memory qubits and 4 distributed compute and distillation blocks.

Spatial Organization of Compute and Distillation Blocks. To achieve a target throughput of one magic state per cycle per compute block, we group compute blocks with multiple distillation units to form a "super-block." This organization is motivated by the fact that individual distillation protocols often exhibit lower throughput than the consumption rate of a compute block [52]. These super-blocks are distributed around the central memory qubits to minimize movement overhead and routing congestion, while ensuring every compute block can access any logical qubit via shared ancilla paths. Figure 16 demonstrates this layout for the 18-qubit QFT.

This strategy is informed by the broader principle that modern architectural optimizations have significantly reduced the resource cost per magic state [52]. By treating the factory cost as a primary variable in our volume-minimization strategy rather than a static overhead, TACO can adaptively scale distillation resources. This enables higher throughput and shorter execution times while maintaining a favorable balance between hardware investment and operational speed.

F. Physical Realization of the Proposed Architecture

The proposed architecture can be physically realized as a specialized lattice-surgery organization built on standard surface-code patches, similar to prior compact, intermediate, and fast FTQC organizations [51]. The key difference is that our organization is tailored to the high locality exposed by TACO-optimized circuits.

We distinguish between *compute* regions and *memory* regions. This distinction is architectural rather than technological: both regions use the same surface-code substrate, physical qubits, and couplers, but serve different logical roles during execution. Compute regions handle active non-Clifford processing, while memory regions primarily store logical states and support Clifford operations. To support arbitrary $\pi/4$ rotations, compute regions employ a modified logical patch that has the same code distance d while expanding the physical footprint from the standard $d\times d$ patch to approximately $2d\times d$, requiring roughly $2\times$ more physical qubits per logical patch. In contrast, memory regions retain the standard patch structure.

Under this organization, qubits are moved between memory and compute regions by teleporting or swapping the logical state between physical patch locations, while the underlying physical qubits remain unchanged. This can be implemented using standard lattice-surgery primitives.

V. Evaluation Methodology

A. Figure of Merit

The primary metric used to evaluate TACO is the reduction in Clifford gate counts. We specifically measure the percentage decrease in CNOT, Hadamard (H), and Phase (S) gates. Additionally, we estimate potential execution time savings by calculating QEC cycle reductions based on the logical gate latencies detailed in Section II-B.

B. QEC Cycle Calculation

We evaluate the temporal overhead of FTQC circuits using two complementary methodologies to capture both theoretical lower bounds and practical architectural constraints.

Architecture-Independent Analysis. We first calculate the total QEC cycles as the critical path length, utilizing the cycles-per-gate values from Section II-B. This metric represents the theoretical minimum cycle count based on the longest dependency chain, independent of hardware resource limitations or routing overhead.

Cycle-Accurate Simulation and Routing. To capture realistic architectural constraints, we use a cycle-accurate simulator that processes the circuit layer by layer. While local operations are applied directly, non-local gates (CNOT) and state-injections (T, S) are routed to available compute blocks. By default, our simulator employs **greedy routing**, which allocates resources to ready gates on a first-come, first-served basis. As a specialized case study demonstrating compatibility with advanced resource management, we integrate **LSQECC routing** [76]. This dual-routing approach validates TACO's performance across varying architectural sophistication.

C. Benchmark Circuits

We evaluate TACO using a diverse set of benchmarks categorized by their structural characteristics. First, we select key algorithms from QASMBench [50], including the Quantum Fourier Transform (`qft`), Quantum Phase Estimation (`qpe`), `ising` model simulation, and `w_state` preparation.

Second, we include four Toffoli-heavy FTQC algorithms from Op-T-mize [48]: `adder`, `csla_mux`, `hwb`, and `qcla_mod`. These represent essential arithmetic subroutines for high-level algorithms such as Shor's algorithm [67]. Table I summarizes the qubit and gate counts for these benchmarks.

TABLE I
BENCHMARK CIRCUIT CHARACTERISTICS.

	qft	ising	qpe	w_state	adder	csla_mux	hwb	qcla_mod
Qubits	18	26	9	76	24	15	16	26
Gates	783	307	36	378	330	70	31764	294
Clifford+T Gates	95968	25520	7587	38223	1128	210	91642	1120
Increase (×)	122.6	83.1	210.8	101.1	3.4	3.0	2.9	3.8

Scalability and Compatibility. To evaluate scalability, we utilize large-scale `qft` circuits ranging from 100 to 300 qubits. Furthermore, to demonstrate that TACO is complementary to existing FTQC synthesis frameworks, we conduct a case study on circuits synthesized by `Synthetiq` [59] and `TRASYN` [39]. We apply TACO as a post-processing step to these outputs to further optimize final gate counts.

Expansion to Clifford+T Basis. Table I reports both the original gate counts (**Gates**) in native representation and compiled counts in the Clifford+T basis. Notably, rotation-heavy circuits (e.g., `qft`, `qpe`) exhibit an expansion of over $83\times$, while Toffoli-dominated circuits experience a smaller growth of 3–$4\times$. This disparity arises from the decomposition of high-level operations: a single rotation gate is synthesized into hundreds of Clifford+T gates, of which roughly 60% are Clifford and 40% are T gates. In contrast, a Toffoli gate can be decomposed into 8 Clifford gates and 7 T gates. Consequently, in rotation-dominated circuits, over 99% of the Clifford+T gates originate from rotation synthesis, whereas for Toffoli-heavy circuits, $\sim 70\%$ result from Toffoli decomposition.

D. Clifford+T Synthesis

We compare TACO against the native Clifford+T transpilation functionality in Qiskit 2.2.3 (Optimization Level 3). While Qiskit employs the Solovay-Kitaev algorithm [24] for approximate unitary synthesis, TACO leverages the GridSynth algorithm [64] for optimal T-count synthesis. For all cases, we set a default synthesis error tolerance of $\epsilon = 10^{-10}$.

VI. Evaluation Results

A. Clifford Gate Reduction

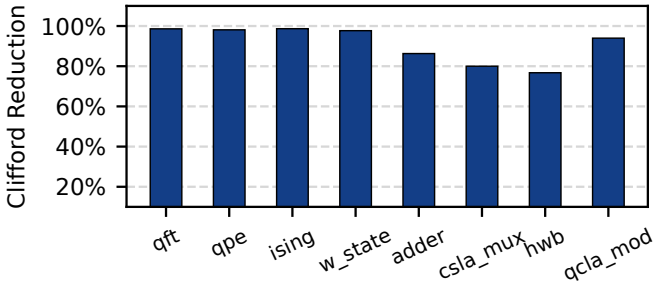


Fig. 17. Clifford gate reduction percentage across benchmark circuits. TACO achieves an average 91.2% Clifford gate reduction across all benchmarks.

Figure 17 presents Clifford gate reduction results for all benchmark circuits. TACO achieves an average Clifford gate reduction of 91.2% and up to 98.6%, highlighting the strength of our optimization. Notably, QFT and QPE circuits show reductions of 98.6% and 98.1%, respectively. The smallest reduction is observed in the hwb circuit at 77%, due to its high initial CNOT gate content (over 65%). Since TACO preserves CNOT gates to maintain gate parallelism, the potential for Clifford reduction in hwb is inherently limited. Nevertheless, as shown in Section VI-B, retaining these CNOT gates ultimately improves overall execution runtime.

B. Speedup of TACO over PBC

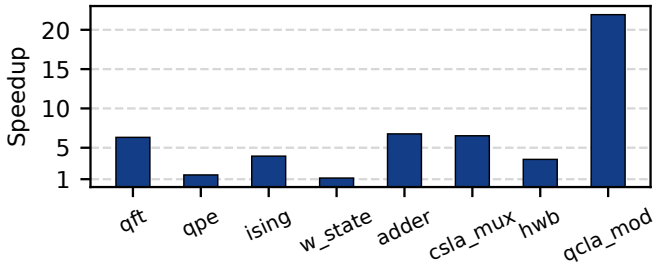


Fig. 18. TACO achieves 1.14× to 21.9× speedup across benchmark circuits over PBC, with a geometric mean speedup of 4.4×.

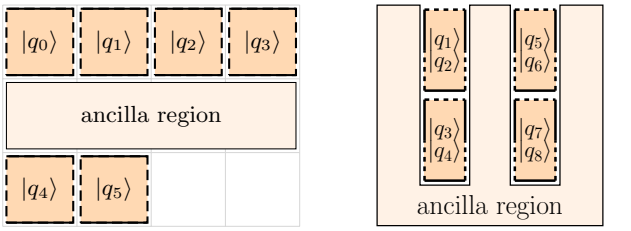
With enhanced gate parallelism, TACO achieves significant speedup over PBC across the benchmarks as shown in Figure 18. On average, TACO achieves 4.4× speedup over the baseline approach. The smallest speedup is observed for w_state at 1.14×, which can be attributed to the circuit's limited CNOT gate density of only 1%. As discussed in Section III-B, the primary source of limited parallelism stems from commuting CNOT gates through the circuit. Since w_state contains relatively few CNOT gates, its circuit depth remains comparable between PBC and TACO, resulting in similar QEC runtime with only modest improvement. Interestingly, the hwb circuit, which has a high CNOT ratio and

shows the worst Clifford gate reduction results, still achieves a 3.52× speedup, demonstrating that runtime performance can be significantly improved even when gate-reduction opportunities are limited. For all remaining benchmark circuits, TACO achieves at least 1.54× speedup, with a maximum of 21.9× for circuits with higher CNOT gate density and greater opportunities for parallel execution.

C. TACO versus PBC: Overall FTQC Volume Reduction

In this section, we perform a comprehensive FTQC resource estimation comparison between PBC and TACO using the 18-qubit QFT circuit. Both approaches produce optimized circuits with 40,777 T gates. While PBC eliminates all Clifford gates, TACO retains 1,080 Clifford gates, resulting in total gate counts of 40,777 and 41,857 gates, respectively. Despite this slightly higher gate count, the key advantage of TACO lies in execution parallelism: it achieves a circuit depth of 6,598 compared to PBC's 39,055, making TACO 5.9× shallower and dramatically reducing the total execution cycles required.

1) Resource Estimation Methodology: We evaluate resource requirements using established FTQC estimation methods [32], [34], [51]. We assume a physical error rate of 10^{-3} and compare two PBC architectures: a compact, area-optimized design and a fast design. We select distillation protocols targeting T-gate error below 1%, with code distance chosen to maintain logical error below 1%. These parameters are standard in FTQC resource estimations [32], [51].



(a) Compact FTQC Architecture. (b) Fast FTQC Architecture.

Fig. 19. PBC architectures for comparison.

2) PBC Architecture: We compare TACO against PBC using two FTQC architectures from prior work [51]. In PBC, multi-qubit $\frac{\pi}{4}$ rotations are executed via lattice surgery between logical qubits and magic state ancillae. Each $\frac{\pi}{4}$ rotation acts on logical edges set by the Pauli string. For example, a `ZZZY` Pauli string requires lattice surgery on the Z edges of qubits 0-2 and both X and Z edges of qubit 3.

Figure 19 shows the two baseline architectures. The 'Compact design' uses $1.5n+3$ qubit tiles but exposes only one edge per qubit to the ancilla region. When the required edges are not exposed, qubits must be rotated before surgery can proceed. Y-edge operations are particularly costly, requiring simultaneous exposure of both X and Z edges. This necessitates three additional ancilla qubits and up to 9 cycles per multi-qubit $\frac{\pi}{4}$ rotation. The 'Fast architecture' exposes all edges of all qubits, enabling one multi-qubit $\frac{\pi}{4}$ rotation per cycle at the cost of significantly more tiles $(2n + \sqrt{8n} + 1)$.

TABLE II
RESOURCE COMPARISON FOR 20-QUBIT QFT CIRCUIT

Metric	PBC Compact	PBC Fast	TACO
Magic state blocks	1	11	33
Magic state tiles	11	121	363
Data tiles	30	49	41
Total tiles	41	170	404
QEC cycles	28,783,535	2,382,355	760,901
Code distance	21	19	19
Physical qubits/tile	882	722	722
Volume (data)	$7.6 \cdot 10^{11}$	$8.4 \cdot 10^{10}$	$2.3 \cdot 10^{10}$
Volume (MSD)	$2.8 \cdot 10^{11}$	$2.1 \cdot 10^{11}$	$2.0 \cdot 10^{11}$
Total volume	$1.0 \cdot 10^{12}$	$2.9 \cdot 10^{11}$	$2.2 \cdot 10^{11}$
Reduction with TACO	**-79%**	**-24%**	-
QEC cycles w. MSC	28,783,535	2,382,355	595,604
Volume (MSC)	$2.8 \cdot 10^{10}$	$2.1 \cdot 10^{10}$	$2.1 \cdot 10^{10}$
Total volume w. MSC	$7.9 \cdot 10^{11}$	$1.1 \cdot 10^{11}$	$3.8 \cdot 10^{10}$
Reduction with TACO	**-95%**	**-63%**	-

3) Distillation Protocol and Throughput: With 4×10^4 T gates, each magic state requires an error rate below $0.01/(4 \times 10^4) = 2.5 \times 10^{-7}$. We employ the standard 15-to-1 distillation protocol, which suppresses errors by $35p^3$ using 11 logical qubit tiles and produces one magic state every 11 cycles [51]. At a physical error rate of 10^{-3}, this yields magic states with error rates of $35 \times (10^{-3})^3 = 3.5 \times 10^{-8}$.

The required distillation blocks depend on each architecture's magic state consumption rate. PBC Compact consumes one magic state every nine cycles, requiring only one distillation block, totaling 28,783,535 QEC cycles. PBC Fast consumes one magic state per cycle, requiring 11 distillation blocks to meet demand and totaling 2,382,355 cycles. With magic state distillation blocks, the optimal number of blocks is found to be 3, each containing 11 distillation units. Using the cycle simulator (discussed in Section V-B), TACO completes execution in 760,901 cycles. This results in total logical qubit tile counts of: PBC Compact (30 data + 11 distillation = 41), PBC Fast (49 data + 121 distillation = 170), and TACO (29 data + 12 compute + 363 distillation = 404).

4) Code Distance: The logical error rate per logical qubit per code cycle can be approximated as $p_L = 0.1(100p)^{(d+1)/2}$ [29], where p is the physical error rate and d is the code distance. To maintain overall logical error below 1%, the code distance must satisfy: total_tiles × total_cycles × $d \times p_L < 0.01$. Given the tile and cycle counts, PBC Compact requires a minimum code distance of 21, while PBC Fast and TACO require only 19 due to shorter execution time. The physical qubits per logical qubit is calculated as $2 * d^2$.

5) Resource Comparison: Table II summarizes the complete resource breakdown across all three architectures. The results demonstrate that TACO significantly reduces qubit-cycle volume and achieves **79%** and **24%** reductions over PBC Compact and Fast, respectively.

Moreover, when using more efficient magic state cultivation (MSC), TACO's optimal architecture containing 4 compute & distillation blocks reduces QEC cycles to 595,604. It requires the same distance-19 code, with the volume for magic states reduced by an order of magnitude [34] compared to magic-state distillation. The result is TACO reduces the qubit-cycle

volume by more than **95%** and **63%** compared to PBC Compact and PBC Fast, respectively.

It's worth noting that, although TACO uses more logical qubit tiles, its overall overhead is significantly lower than that of PBC-based approaches. More importantly, between the two PBC-based approaches, the fast design is more efficient than the compact design. PBC Compact's data volume overhead is 2.7× higher than magic state volume, while PBC Fast has comparable overhead. By adopting TACO, the magic-state overhead remains nearly constant while the data volume is significantly reduced, leading to an overall reduction. In this context, further reducing T gate overhead in PBC-based architectures has diminishing returns, whereas in TACO, such optimizations can be more impactful. Thus, TACO improves the effectiveness of existing T gate optimizations.

6) Sensitivity to Magic-State Throughput: We further examine the same case study under different magic-state throughputs. In the default TACO configuration, the system provisions enough factories to supply 4 magic states per round, resulting in 64.5k physical qubits, 595k QEC cycles, and a 2.7× reduction in FTQC execution overhead measured as qubit-cycle volume relative to PBC Fast, which uses 44k physical qubits. Reducing the throughput to 3 magic states per round lowers the physical-qubit count to 47.6k, which is only slightly above PBC Fast, while increasing runtime to 760k QEC cycles. Reducing further to 2 magic states per round lowers the physical-qubit count to 30.8k, which is below PBC Fast, but increases runtime to 1.14M QEC cycles. Despite this area–time tradeoff, TACO still achieves 2.9× and 2.99× lower qubit-cycle volume than PBC Fast, respectively, as shown in Table III. This shows that the benefit of TACO is preserved even under tighter qubit budgets.

TABLE III
TACO UNDER REDUCED MAGIC-STATE THROUGHPUT.

TACO config.	Phys. qubits	QEC cycles	Vol. red. vs. PBC Fast
4 magic states / round	64.5k	595k	2.70×
3 magic states / round	47.6k	760k	2.90×
2 magic states / round	30.8k	1.14M	2.99×

D. TACO Combined with LSQECC and EDPC

To demonstrate that the Clifford-reduced circuits produced by TACO can benefit from orthogonal routing optimizations, we compile the 20-qubit QFT circuit optimized by TACO using LSQECC [76]. Since LSQECC currently does not support the TACO layout with compute sites, we use its default EDPC layout [49]. Following the same methodology described in Section VI-C, we obtain a final FTQC volume of 1.33×10^{11} with magic state distillation, corresponding to a further 1.7× reduction compared to the naive routing baseline (Table IV). We expect further reductions in total FTQC volume once LSQECC supports TACO layout with compute sites as the high locality of the circuit can significantly reduce the number of operations required for routing.

TABLE IV
IMPROVED FTQC VOLUME OF TACO WITH LSQECC ROUTING.

Routing Method	FTQC Volume	Reduction
Naive (greedy)	2.2×10^{11}	Baseline
LSQECC (EDPC)	1.33×10^{11}	1.7×

These results confirm that TACO and routing optimizations target complementary layers of the compilation stack. While routing algorithms improve spatial scheduling, TACO focuses on circuit-level simplification that enables compatibility with such optimizations. In contrast, PBC offers no routing opportunities since each layer contains only a single operation.

TABLE V
LSQECC EXECUTION TIME FOR QFT.

	LSQECC	LSQECC + TACO
Lattice-Surgery Slices	123,139	62,503

On the other hand, TACO not only benefits from LSQECC's routing algorithm, but also improves LSQECC execution through its Clifford reduction. To quantify this effect, we run LSQECC on both the original and the TACO-optimized QFT circuits, using the same layout and magic-state factory configuration (i.e., the same physical-qubit budget). As shown in Table V, the original QFT circuit requires 123,139 lattice-surgery slices, whereas the TACO-optimized circuit requires only 62,503 slices. This corresponds to a nearly 2× reduction in execution time. These results demonstrate that TACO and LSQECC are synergistic: LSQECC improves the routing of TACO-optimized circuits, while TACO reduces the execution time of LSQECC by simplifying the underlying circuit.

E. TACO: Scalability

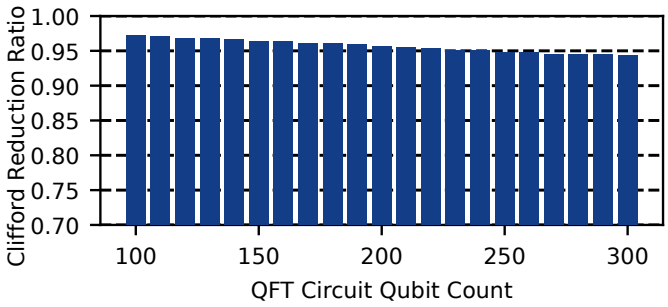


Fig. 20. Clifford gate reduction ratios achieved by TACO on QFT circuits ranging from 100 to 300 qubits. The reduction ratio remains consistently around 95% across all circuit sizes, demonstrating the scalability of TACO to large-scale fault-tolerant quantum circuits.

TACO reduces Clifford gates via local optimizations, specifically by simplifying Clifford operations in synthesized gate sequences of single-qubit R_z rotations and in the decomposition of CCX gates. Since these two sources dominate the Clifford gate count in FTQC circuits, the reduction achieved by TACO naturally extends to larger circuits.

To evaluate scalability, we apply TACO to QFT circuits ranging from 100 to 300 qubits and measure the resulting Clifford reduction ratios. As shown in Figure 20, the reduction

ratio remains consistently around **95%** across all tested sizes, demonstrating the scalability of TACO to large circuits while maintaining high optimization efficiency.

F. Clifford+T Transpilation

TABLE VI
CLIFFORD+T TRANSPILATION COMPARISON BETWEEN QISKIT AND TACO

	Qiskit-O3			TACO		
Circuit	**Unitaries to Synthesize**	**T Gates**	**Time (s)**	**Unitaries to Synthesize**	**T Gates**	**Time (s)**
QFT	408	2,623,881	34.73	378	9,529	0.041
QPE	30	275,356	3.39	30	411	0.013
Ising	75	589,669	7.69	75	1,500	0.049
W-State	148	1,383,832	18.85	75	2,218	0.121
TACO Improvement over Qiskit				*1.26×*	*490×*	*352×*

We compare TACO against Qiskit for Clifford+T transpilation. We focus on three key metrics: the number of unitaries requiring synthesis, the number of T gates, and transpilation time. Of the eight benchmark circuits, the first four (qft, qpe, ising, and w_state) include arbitrary rotation gates that require synthesis, while the others can be trivially decomposed into Clifford+T gates. Therefore, we only include comparisons for the non-trivial benchmarks. Results are shown in Table VI. TACO reduces the number of unitaries requiring synthesis by 1.26× on average, even compared to Qiskit with O3 optimization. TACO achieves 490× fewer T gates on average, which is attributed to both fewer unitaries requiring synthesis and the use of a more efficient synthesis algorithm. More importantly, TACO achieves these superior results with an average 352× faster transpilation time.

G. Applicability to Prior Clifford+T Synthesis

Synthesizing algorithmic circuits into fault-tolerant Clifford+T circuits is an important step in the FTQC toolchain, and several prior works have made significant progress on this problem, including Synthetiq [59] and TRASYN [39]. These approaches are orthogonal to TACO: while they synthesize algorithmic circuits into Clifford+T form, TACO operates *after* synthesis and further reduces the remaining Clifford overhead in the resulting circuits. One possible concern, however, is that synthesized Clifford+T circuits may exhibit less regular structure, potentially making the Clifford-reduction techniques used by TACO more challenging to apply. To evaluate the robustness of TACO in this setting, we collect the synthesized benchmark circuits reported in the original Synthetiq and TRASYN papers and apply TACO to them. For each comparison, we ask two questions: (1) whether Clifford gates still dominate the total gate count after synthesis, and (2) how much of those Clifford gates can be eliminated by TACO.

Figure 21 shows the results on the 16 synthesized Clifford+T circuits reported by Synthetiq. Although these benchmarks are fairly small, ranging from 10 to 68 total Clifford+T gates, Clifford gates still dominate every synthesized circuit, with a minimum Clifford ratio of 66.7% and a median of 76.5%. This confirms that even after synthesis, Clifford gates remain the majority component. At the same time, because Synthetiq is designed to synthesize individual operations, the resulting circuits are relatively small, leaving less room for further optimization. Even in this constrained setting, TACO still removes a substantial fraction of the Clifford gates, achieving 49.1% reduction on average, 41.0% at the median, and up to 92.9% in the best case.

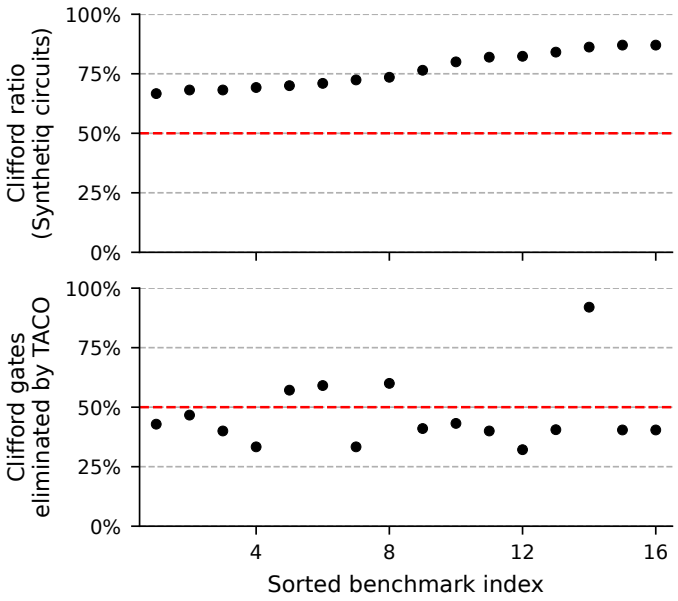


Fig. 21. Across 16 synthesized Clifford+T circuits from Synthetiq, whose sizes range from 10 to 68 total Clifford+T gates, Clifford gates remain the majority after synthesis and TACO still eliminates a substantial fraction of them. **Top:** Clifford-gate ratio in each synthesized circuit. **Bottom:** fraction of Clifford gates eliminated by TACO on the same circuit. Benchmarks are sorted in ascending order of Clifford ratio in the top panel, and the bottom panel follows the same order for direct correspondence.

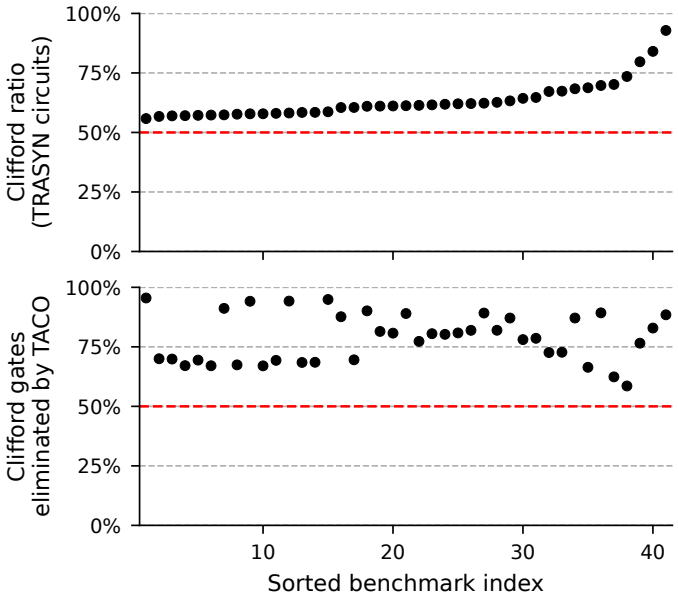


Fig. 22. Across 41 synthesized Clifford+T circuits from TRASYN, whose sizes range from 28 to 26,215 total Clifford+T gates, Clifford gates remain the majority after synthesis and TACO still eliminates a substantial fraction of them. **Top:** Clifford-gate ratio in each synthesized circuit. **Bottom:** fraction of Clifford gates eliminated by TACO on the same circuit. Benchmarks are sorted in ascending order of Clifford ratio in the top panel, and the bottom panel follows the same order for direct correspondence.

Figure 22 shows the corresponding results on the 41 synthesized Clifford+T circuits from TRASYN. In contrast to Synthetiq, TRASYN performs synthesis at the circuit level and therefore produces benchmarks that are much larger overall, ranging from 28 to 26,215 total Clifford+T gates. Nevertheless, Clifford gates still remain the majority in all synthesized circuits, with a minimum Clifford ratio of 55.8% and a median of 61.2%. More importantly, TACO continues to achieve strong reductions in this larger-scale setting, eliminating 78.7% of Clifford gates on average and 80.2% at the median, with a maximum reduction of 95.5%. This reduction is much closer to the savings achieved on our own benchmark suite, indicating that TACO remains highly effective even when applied to optimized Clifford+T circuits produced by state-of-the-art circuit synthesis works.

Overall, these results show that prior synthesis and TACO are complementary rather than competing. Existing tools such as Synthetiq and TRASYN reduce the cost of converting algorithmic descriptions into Clifford+T form, while TACO can be applied afterward to further reduce the Clifford overhead that still dominates the synthesized circuits.

VII. Related Work

A. Pauli-Based Computation

PBC is a framework for analyzing and optimizing quantum circuits by representing quantum gates as Pauli rotation operations and studying their commutation relationships through Pauli strings. Different works have employed this framework for various purposes: Litinski used PBC to eliminate Clifford gates in fault-tolerant architectures [51], while others have applied PBC to reduce resources in NISQ circuits [62], to extend the formalism to higher-dimensional systems [61], or to design PBC-inspired optimization methods [60]. However, these non-FTQC optimizations are not applicable to FTQC, as they introduce additional operations that cannot be executed fault-tolerantly. Therefore, the only relevant prior work to our setting is Litinski's PBC-based approach for eliminating Clifford gates. While PBC is effective when T-magic-state generation is the bottleneck [51], recent advances [34], [52] have lowered T-state costs, shifting the critical challenge to circuit parallelism. Thus, TACO's ability to eliminate Clifford overhead while preserving parallelism is essential for realizing the quantum speedups envisioned in prior work [32].

B. QEC Codes and Optimization

This paper focuses on patch-based surface code with lattice surgery. Other QEC constructions include defect-based [30], [42], [51] and twist-based [9] surface code. Overlapping multiple surface code patches on neutral atom devices was also explored [74]. Beyond surface codes, quantum stabilizer codes like Shor's [4], [66] and Steane's [69] use stabilizer groups to define protected logical subspaces. Color codes [10] support transversal Clifford operations and have been experimentally shown [8], [41], [58], [65]. Quantum LDPC codes [11], [17], [22] feature long-range checks and higher code rates. Fault-tolerant non-Clifford operations require magic states. Various distillation protocols have been proposed to prepare high-fidelity magic states using Reed-Muller code [13], [37], block codes [12], [28], [44], and recent optimizations [20], [32], [34], [52]. QEC decoding uses syndrome measurements to identify errors through lookup tables [23], [71], MWPM [2], [25], [26], [40], [68], [75], [77], or machine learning [3], [5], [21]. System optimizations for synchronizing lattice surgery [56] and speculative decoding [73] have also been explored.

C. General Clifford Circuit Optimization

Several works have focused on optimizing circuits containing Clifford gates. Maslov et al. [54] and Bravyi et al. [14] employ template-based circuit optimization techniques, which attempt to replace sequences of Clifford gates with more efficient alternatives. However, in Clifford+T circuits, the consecutive Clifford-only gates sandwiched between T gates are typically very short (usually fewer than 3 gates), making template-matching techniques ineffective. In contrast, TACO studies how to eliminate Clifford gates in the presence of non-Clifford gates. Liu et al. [53] proposed absorbing Clifford gates into neighboring gates, resulting in new composite gates. While such gates might be executable in NISQ devices, they cannot be fault-tolerantly executed under QEC schemes.

VIII. Conclusion

Optimizing Clifford gates remains a key challenge for enabling practical FTQC. We present TACO, a Transpiler-Architecture Co-design Optimization framework that closes this gap, achieving an average 91.2% reduction in Clifford gates across a range of quantum circuits while fully preserving gate parallelism. TACO delivers up to 21.9× speedup over PBC, with a geometric mean speedup of 4.4× across benchmarks. Our co-designed FTQC architecture further boosts efficiency by exploiting the locality of optimized rotation sequences, enabling one logical gate per QEC cycle with just $1.5n + 4$ logical qubit tiles. Together, these cross-stack optimizations at both circuit and architectural levels mark a significant advance toward efficient, practical FTQC.

Acknowledgment

This material is based upon work supported by the U.S. Department of Energy, Office of Science, National Quantum Information Science Research Centers, Co-design Center for Quantum Advantage (C2QA) under contract number DE-SC0012704 (Basic Energy Sciences, PNNL FWP 76274). This research was also supported in part by the National Research Council (NRC) Canada grants AQC 003 and AQC 213, as well as the Natural Sciences and Engineering Research Council of Canada (NSERC) [funding number RGPIN-2019-05059]. This research used resources and associated infrastructure support of the Oak Ridge Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC05-00OR22725. This research used resources of the National Energy Research Scientific Computing Center (NERSC), a U.S. Department of Energy Office of Science User Facility located at Lawrence Berkeley National Laboratory, operated under Contract No. DE-AC02-05CH11231.

APPENDIX: ARTIFACT EVALUATION

A. Abstract

This artifact provides the Zenodo reproducibility package for the **TACO** framework and the experimental workflow used in the paper. It contains the benchmark datasets, figure-generation scripts, and a self-contained copy of the NWQEC codebase with TACO functionality integrated. The artifact is available at https://doi.org/10.5281/zenodo.19449157, and NWQEC is open-sourced on GitHub at https://github.com/pnnl/nwqec. The package reproduces key results on Clifford reduction and gate parallelism via automated command-line workflows that generate intermediate CSV files and final PDF figures. All experiments run on a standard CPU machine without specialized hardware and complete within tens of minutes. Detailed build and execution instructions are provided in the artifact `README`.

B. Artifact check-list (meta-information)

- **Algorithm:** Clifford+T and Pauli-based computation (PBC) FTQC transpilation; Clifford-reduction analysis; gate-parallelism analysis
- **Program:** NWQEC (`nwqec-cli`), Bash workflows, Python plotting/analysis scripts
- **Compilation:** CMake + C++17
- **Transformations:** N/A
- **Binary:** `nwqec-cli`
- **Data set:** QASM benchmark circuits (paper benchmark set + prior-work benchmark sets)
- **Run-time environment:** Linux or macOS shell environment with Python 3
- **Hardware:** CPU-only
- **Execution:** Scripted command-line workflows via top-level Bash scripts
- **Metrics:** Clifford ratio, Clifford reduction ratio, gate parallelism, and operation-weight distributions
- **Output:** CSV files in `results/` and PDF figures in `figures/`
- **Experiments:** Fig. 5, Fig. 14, Fig. 17, Fig. 20, Fig. 21, Fig. 22
- **How much disk space required (approximately)?:** $<$ 100 MB
- **How much time is needed to prepare workflow (approximately)?:** 5–10 minutes (build + environment setup)
- **How much time is needed to complete experiments (approximately)?:** 10–20 minutes (Fig. 20 is the longest)
- **Publicly available?:** yes
- **Code licenses (if publicly available)?:** MIT
- **Data licenses (if publicly available)?:** MIT
- **Workflow automation framework used?:** Bash scripts
- **Archived (provide DOI)?:** 10.5281/zenodo.19449157

C. Description

1) How to access: The artifact can be downloaded from the DOI link: https://doi.org/10.5281/zenodo.19449157

2) Hardware dependencies: No specialized hardware is required. All experiments run on a standard CPU-based workstation or laptop.

3) Software dependencies:

- CMake and a C++17 compiler
- GMP and MPFR, used by the synthesis backend. On macOS and Linux, these dependencies need not be installed manually: NWQEC automatically uses precompiled binaries, downloading them if they are not already present
- Python 3 with plotting/data packages used by scripts

This artifact has been tested on macOS and Linux.

4) Data sets: Benchmark circuits are included:

- `benchmarks/`: benchmark circuits used in the main paper experiments
- `synthetiq/`: circuits generated by the Synthetiq benchmark suite
- `trasyn/`: circuits generated by the TRASYN benchmark suite

5) Models: N/A.

D. Installation

Build NWQEC from the artifact root directory:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
```

E. Experiment workflow

The artifact provides automated scripts to reproduce the experimental results reported in the paper.

For convenience, the full workflow can be executed with a single script:

```
./run_all.sh
```

This script performs a fresh build of the NWQEC binary, runs all experiments on the benchmark circuits, collects metrics into CSV files under `results/`, and generates the corresponding PDF figures under `figures/`.

Individual figures can also be reproduced using dedicated top-level scripts (one per figure). For example:

```
./plot_fig_5.sh
```

Each figure script **both runs the necessary experiment and generates the final plotted figure**. If the corresponding CSV results already exist, the experiment is skipped to save time, and only the figure is generated. Passing the `--force-collect` flag forces the script to re-run the experiment and regenerate the data.

Detailed usage instructions and script options are provided in the artifact `README`.

F. Evaluation and expected results

The artifact should regenerate the following figure PDFs in `figures/`:

- Fig. 5: QFT operation-weight distribution
- Fig. 14: QFT gate parallelism
- Fig. 17: Clifford reduction on benchmark set
- Fig. 20: large-QFT reduction trend
- Fig. 21: Synthetiq comparison
- Fig. 22: TRASYN comparison

Numeric outputs are generated as CSV files in `results/`. The figures should match trends and relative values reported in the paper. Minor numerical or formatting differences may occur due to environment or library variations.

G. Experiment customization

Scripts support standard environment-variable overrides for the Python interpreter and the NWQEC binary path. The README documents available script flags for forced regeneration and figure-specific customization.

H. Notes

The artifact is designed to reproduce the reported trends and relative improvements rather than to serve as a complete benchmark suite. Generated outputs may differ slightly across environments due to library versions, compiler settings, or plotting backends.

I. Methodology

Submission, reviewing, and badging methodology:

- https://www.acm.org/publications/policies/artifact-review-and-badging-current
- https://cTuning.org/ae

References

[1] "Suppressing quantum errors by scaling a surface code logical qubit," *Nature*, vol. 614, no. 7949, pp. 676–681, 2023.

[2] N. Alavisamani, S. Vittal, R. Ayanzadeh, P. Das, and M. Qureshi, "Promatch: Extending the reach of real-time quantum error correction with adaptive predecoding," 2024. [Online]. Available: https://arxiv.org/abs/2404.03136

[3] P. Andreasson, J. Johansson, S. Liljestrand, and M. Granath, "Quantum error correction for the toric code using deep reinforcement learning," *Quantum*, vol. 3, p. 183, Sep. 2019. [Online]. Available: https://doi.org/10.22331/q-2019-09-02-183

[4] D. Bacon, "Operator quantum error-correcting subsystems for self-correcting quantum memories," *Phys. Rev. A*, vol. 73, p. 012340, Jan 2006. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevA.73.012340

[5] P. Baireuther, M. D. Caio, B. Criger, C. W. J. Beenakker, and T. E. O'Brien, "Neural network decoder for topological color codes with circuit level noise," *New Journal of Physics*, vol. 21, no. 1, p. 013003, jan 2019. [Online]. Available: https://dx.doi.org/10.1088/1367-2630/aaf29e

[6] V. Bergholm, J. Izaac, M. Schuld, C. Gogolin, S. Ahmed, V. Ajith, M. S. Alam, G. Alonso-Linaje, B. AkashNarayanan, A. Asadi *et al.*, "Pennylane: Automatic differentiation of hybrid quantum-classical computations," *arXiv preprint arXiv:1811.04968*, 2018.

[7] N. S. Blunt, G. P. Gehér, and A. E. Moylett, "Compilation of a simple chemistry application to quantum error correction primitives," *Physical review research*, vol. 6, no. 1, p. 013325, 2024.

[8] D. Bluvstein, S. J. Evered, A. A. Geim, S. H. Li, H. Zhou, T. Manovitz, S. Ebadi, M. Cain, M. Kalinowski, D. Hangleiter *et al.*, "Logical quantum processor based on reconfigurable atom arrays," *Nature*, vol. 626, no. 7997, pp. 58–65, 2024.

[9] H. Bombin, "Topological order with a twist: Ising anyons from an abelian model," *Phys. Rev. Lett.*, vol. 105, p. 030403, Jul 2010. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevLett.105.030403

[10] H. Bombin and M. A. Martin-Delgado, "Topological quantum distillation," *Phys. Rev. Lett.*, vol. 97, p. 180501, Oct 2006. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevLett.97.180501

[11] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, "High-threshold and low-overhead fault-tolerant quantum memory," *Nature*, vol. 627, no. 8005, pp. 778–782, 2024. [Online]. Available: https://doi.org/10.1038/s41586-024-07107-7

[12] S. Bravyi and J. Haah, "Magic-state distillation with low overhead," *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 86, no. 5, p. 052329, 2012.

[13] S. Bravyi and A. Kitaev, "Universal quantum computation with ideal clifford gates and noisy ancillas," *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 71, no. 2, p. 022316, 2005.

[14] S. Bravyi, R. Shaydulin, S. Hu, and D. Maslov, "Clifford circuit optimization with templates and symbolic pauli gates," *Quantum*, vol. 5, p. 580, 2021.

[15] S. Bravyi, G. Smith, and J. A. Smolin, "Trading classical and quantum computational resources," *Phys. Rev. X*, vol. 6, p. 021043, Jun 2016. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevX.6.021043

[16] S. B. Bravyi and A. Y. Kitaev, "Quantum codes on a lattice with boundary," *arXiv preprint quant-ph/9811052*, 1998.

[17] N. P. Breuckmann and J. N. Eberhardt, "Quantum low-density parity-check codes," *PRX Quantum*, vol. 2, p. 040101, Oct 2021. [Online]. Available: https://link.aps.org/doi/10.1103/PRXQuantum.2.040101

[18] A. R. Calderbank and P. W. Shor, "Good quantum error-correcting codes exist," *Physical Review A*, vol. 54, no. 2, p. 1098, 1996.

[19] E. T. Campbell and M. Howard, "Unified framework for magic state distillation and multiqubit gate synthesis with reduced resource cost," *Physical Review A*, vol. 95, no. 2, Feb. 2017. [Online]. Available: http://dx.doi.org/10.1103/PhysRevA.95.022316

[20] E. T. Campbell and M. Howard, "Magic state parity-checker with pre-distilled components," *Quantum*, vol. 2, p. 56, Mar. 2018. [Online]. Available: https://doi.org/10.22331/q-2018-03-14-56

[21] C. Chamberland, L. Goncalves, P. Sivarajah, E. Peterson, and S. Grimberg, "Techniques for combining fast local decoders with global decoders under circuit-level noise," *Quantum Science and Technology*, vol. 8, no. 4, p. 045011, jul 2023. [Online]. Available: https://dx.doi.org/10.1088/2058-9565/ace64d

[22] L. Z. Cohen, I. H. Kim, S. D. Bartlett, and B. J. Brown, "Low-overhead fault-tolerant quantum computing using long-range connectivity," *Science Advances*, vol. 8, no. 20, p. eabn1717, 2022. [Online]. Available: https://www.science.org/doi/abs/10.1126/sciadv.abn1717

[23] P. Das, A. Locharla, and C. Jones, "Lilliput: a lightweight low-latency lookup-table decoder for near-term quantum error correction," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 541–553. [Online]. Available: https://doi.org/10.1145/3503222.3507707

[24] C. M. Dawson and M. A. Nielsen, "The solovay-kitaev algorithm," *arXiv preprint quant-ph/0505030*, 2005.

[25] A. deMarti iOlius, P. Fuentes, R. Orús, P. M. Crespo, and J. Etxezarreta Martinez, "Decoding algorithms for surface codes," *Quantum*, vol. 8, p. 1498, Oct. 2024. [Online]. Available: https://doi.org/10.22331/q-2024-10-10-1498

[26] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, "Topological quantum memory," *Journal of Mathematical Physics*, vol. 43, no. 9, pp. 4452–4505, 2002.

[27] A. Erhard, H. Poulsen Nautrup, M. Meth, L. Postler, R. Stricker, M. Stadler, V. Negnevitsky, M. Ringbauer, P. Schindler, H. J. Briegel *et al.*, "Entangling logical qubits with lattice surgery," *Nature*, vol. 589, no. 7841, pp. 220–224, 2021.

[28] A. G. Fowler, S. J. Devitt, and C. Jones, "Surface code implementation of block code state distillation," *Scientific Reports*, vol. 3, no. 1, p. 1939, 2013. [Online]. Available: https://doi.org/10.1038/srep01939

[29] A. G. Fowler and C. Gidney, "Low overhead quantum computation using lattice surgery," *arXiv preprint arXiv:1808.06709*, 2018.

[30] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 86, no. 3, p. 032324, 2012.

[31] J. M. Gambetta, J. M. Chow, and M. Steffen, "Building logical qubits in a superconducting quantum computing system," *npj quantum information*, vol. 3, no. 1, p. 2, 2017.

[32] C. Gidney and M. Ekerå, "How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits," *Quantum*, vol. 5, p. 433, 2021.

[33] C. Gidney and A. G. Fowler, "Efficient magic state factories with a catalyzed — ccz\rangle to 2|t\rangle transformation," *Quantum*, vol. 3, p. 135, 2019.

[34] C. Gidney, N. Shutty, and C. Jones, "Magic state cultivation: growing t states as cheap as cnot gates," 2024. [Online]. Available: https://arxiv.org/abs/2409.17595

[35] B. Giles and P. Selinger, "Remarks on matsumoto and amano's normal form for single-qubit clifford+ t operators," *arXiv preprint arXiv:1312.6584*, 2013.

[36] D. Gottesman, *Stabilizer codes and quantum error correction*. California Institute of Technology, 1997.

[37] J. Haah and M. B. Hastings, "Codes and Protocols for Distilling T, controlled-S, and Toffoli Gates," *Quantum*, vol. 2, p. 71, Jun. 2018. [Online]. Available: https://doi.org/10.22331/q-2018-06-07-71

[38] J. Haah, M. B. Hastings, D. Poulin, and D. Wecker, "Magic state distillation with low space overhead and optimal asymptotic input count," *Quantum*, vol. 1, p. 31, 2017.

[39] T. Hao, A. Xu, and S. Tannu, "Reducing t gates with unitary synthesis," *arXiv preprint arXiv:2503.15843*, 2025.

[40] O. Higgott and C. Gidney, "Sparse blossom: correcting a million errors per core second with minimum-weight matching," 2023. [Online]. Available: https://arxiv.org/abs/2303.15933

[41] J. Hilder, D. Pijn, O. Onishchenko, A. Stahl, M. Orth, B. Lekitsch, A. Rodriguez-Blanco, M. Müller, F. Schmidt-Kaler, and U. G. Poschinger, "Fault-tolerant parity readout on a shuttling-based trapped-ion quantum computer," *Phys. Rev. X*, vol. 12, p. 011032, Feb 2022. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevX.12.011032

[42] D. Horsman, A. G. Fowler, S. Devitt, and R. Van Meter, "Surface code quantum computing by lattice surgery," *New Journal of Physics*, vol. 14, no. 12, p. 123011, 2012.

[43] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," 2024.

[44] C. Jones, "Multilevel distillation of magic states for quantum computing," *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 87, no. 4, p. 042305, 2013.

[45] A. Y. Kitaev, "Quantum computations: algorithms and error correction," *Russian Mathematical Surveys*, vol. 52, no. 6, p. 1191, 1997.

[46] A. Y. Kitaev, "Fault-tolerant quantum computation by anyons," *Annals of physics*, vol. 303, no. 1, pp. 2–30, 2003.

[47] E. Knill, R. Laflamme, and G. J. Milburn, "A scheme for efficient quantum computation with linear optics," *nature*, vol. 409, no. 6816, pp. 46–52, 2001.

[48] K. Kottmann, "T-gate optimization," 2024. [Online]. Available: https://pennylane.ai/datasets/op-T-mize

[49] T. LeBlond, C. Dean, G. Watkins, and R. Bennink, "Realistic cost to execute practical quantum circuits using direct clifford+ t lattice surgery compilation," *ACM Transactions on Quantum Computing*, vol. 5, no. 4, pp. 1–28, 2024.

[50] A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, "Qasmbench: A low-level qasm benchmark suite for nisq evaluation and simulation," 2022. [Online]. Available: https://arxiv.org/abs/2005.13018

[51] D. Litinski, "A game of surface codes: Large-scale quantum computing with lattice surgery," *Quantum*, vol. 3, p. 128, 2019.

[52] D. Litinski, "Magic state distillation: Not as costly as you think," *Quantum*, vol. 3, p. 205, 2019.

[53] L. Liu and X. Dou, "Qucloud+: A holistic qubit mapping scheme for single/multi-programming on 2d/3d nisq quantum computers," *ACM Transactions on Architecture and Code Optimization*, vol. 21, no. 1, pp. 1–27, 2024.

[54] D. Maslov, G. W. Dueck, D. M. Miller, and C. Negrevergne, "Quantum circuit simplification and level compaction," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 3, pp. 436–444, 2008.

[55] K. Matsumoto and K. Amano, "Representation of quantum circuits with clifford and pi/8 gates," *arXiv preprint arXiv:0806.3834*, 2008.

[56] S. Maurya and S. Tannu, "Synchronization for fault-tolerant quantum computers," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1370–1385.

[57] M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*. Cambridge university press, 2010.

[58] D. Nigg, M. Müller, E. A. Martinez, P. Schindler, M. Hennrich, T. Monz, M. A. Martin-Delgado, and R. Blatt, "Quantum computations on a topologically encoded qubit," *Science*, vol. 345, no. 6194, pp. 302–305, 2014. [Online]. Available: https://www.science.org/doi/abs/10.1126/science.1253742

[59] A. Paradis, J. Dekoninck, B. Bichsel, and M. Vechev, "Synthetiq: Fast and versatile quantum circuit synthesis," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 55–82, 2024.

[60] J. Paykin, A. T. Schmitz, M. Ibrahim, X.-C. Wu, and A. Y. Matsuura, "Pcoast: A pauli-based quantum circuit optimization framework," in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 01, 2023, pp. 715–726.

[61] F. C. R. Peres, "Pauli-based model of quantum computation with higher-dimensional systems," *Phys. Rev. A*, vol. 108, p. 032606, Sep 2023. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevA.108.032606

[62] F. C. R. Peres and E. F. Galvão, "Quantum circuit compilation and hybrid computation using Pauli-based computation," *Quantum*, vol. 7, p. 1126, oct 2023. [Online]. Available: https://doi.org/10.22331/q-2023-10-03-1126

[63] B. W. Reichardt, A. Paetznick, D. Aasen, I. Basov, J. M. Bello-Rivas, P. Bonderson, R. Chao, W. van Dam, M. B. Hastings, A. Paz, M. P. da Silva, A. Sundaram, K. M. Svore, A. Vaschillo, Z. Wang, M. Zanner, W. B. Cairncross, C.-A. Chen, D. Crow, H. Kim, J. M. Kindem, J. King, M. McDonald, M. A. Norcia, A. Ryou, M. Stone, L. Wadleigh, K. Barnes, P. Battaglino, T. C. Bohdanowicz, G. Booth, A. Brown, M. O. Brown, K. Cassella, R. Coxe, J. M. Epstein, M. Feldkamp, C. Griger, E. Halperin, A. Heinz, F. Hummel, M. Jaffe, A. M. W. Jones, E. Kapit, K. Kotru, J. Lauigan, M. Li, J. Marjanovic, E. Megidish, M. Meredith, R. Morshead, J. A. Muniz, S. Narayanaswami, C. Nishiguchi, T. Paule, K. A. Pawlak, K. L. Pudenz, D. R. Pérez, J. Simon, A. Smull, D. Stack, M. Urbanek, R. J. M. van de Veerdonk, Z. Vendeiro, R. T. Weverka, T. Wilkason, T.-Y. Wu, X. Xie, E. Zalys-Geller, X. Zhang, and B. J. Bloom, "Logical computation demonstrated with a neutral atom quantum processor," 2024. [Online]. Available: https://arxiv.org/abs/2411.11822

[64] N. J. Ross and P. Selinger, "Optimal ancilla-free clifford+ t approximation of z-rotations," *arXiv preprint arXiv:1403.2975*, 2014.

[65] C. Ryan-Anderson, J. G. Bohnet, K. Lee, D. Gresh, A. Hankin, J. P. Gaebler, D. Francois, A. Chernoguzov, D. Lucchetti, N. C. Brown, T. M. Gatterman, S. K. Halit, K. Gilmore, J. A. Gerber, B. Neyenhuis, D. Hayes, and R. P. Stutz, "Realization of real-time fault-tolerant quantum error correction," *Phys. Rev. X*, vol. 11, p. 041058, Dec 2021. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevX.11.041058

[66] P. W. Shor, "Scheme for reducing decoherence in quantum computer memory," *Physical review A*, vol. 52, no. 4, p. R2493, 1995.

[67] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.

[68] S. C. Smith, B. J. Brown, and S. D. Bartlett, "Local predecoder to reduce the bandwidth and latency of quantum error correction," *Phys. Rev. Appl.*, vol. 19, p. 034050, Mar 2023. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevApplied.19.034050

[69] A. Steane, "Multiple-particle interference and quantum error correction," *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, vol. 452, no. 1954, pp. 2551–2577, 1996. [Online]. Available: https://royalsocietypublishing.org/doi/abs/10.1098/rspa.1996.0136

[70] D. B. Tan, M. Y. Niu, and C. Gidney, "A sat scalpel for lattice surgery: Representation and synthesis of subroutines for surface-code fault-tolerant quantum computing," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 325–339.

[71] Y. Tomita and K. M. Svore, "Low-distance surface codes under realistic quantum noise," *Phys. Rev. A*, vol. 90, p. 062320, Dec 2014. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevA.90.062320

[72] J. J. Vartiainen, M. Möttönen, and M. M. Salomaa, "Efficient decomposition of quantum gates," *Physical review letters*, vol. 92, no. 17, p. 177902, 2004.

[73] J. Viszlai, J. D. Chadwick, S. Joshi, G. S. Ravi, Y. Li, and F. T. Chong, "Swiper: Minimizing fault-tolerant quantum program latency via speculative window decoding," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1386–1401.

[74] J. Viszlai, S. Lin, S. Dangwal, C. Bradley, V. Ramesh, J. Baker, H. Bernien, and F. T. Chong, "Interleaved logical qubits in atom arrays," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 261–274.

[75] S. Vittal, P. Das, and M. Qureshi, "Astrea: Accurate quantum error-decoding via practical minimum-weight perfect-matching," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3579371.3589037

[76] G. Watkins, H. M. Nguyen, K. Watkins, S. Pearce, H.-K. Lau, and A. Paler, "A high performance compiler for very large scale surface code computations," *Quantum*, vol. 8, p. 1354, 2024.

[77] Y. Wu and L. Zhong, "Fusion blossom: Fast mwpm decoders for qec," 2023. [Online]. Available: https://arxiv.org/abs/2305.08307